COEP TECHNOLOGICAL UNIVERSITY · DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mini Project 5: Blockchain-Lite Ledger & Digital Wallet

Foundations-First Framework (Python 3.11+ Standard Library Only) · Protocol Spec, HLD, Statistical PoW Analysis & Complete Viva Guide

1. Executive Summary & Four-Layer Foundations-First Architecture

This project implements a complete, tamper-evident, UTXO-based **Blockchain-Lite Ledger & Digital Wallet** from mathematical and algorithmic first principles using only the Python 3.11+ standard library (`CON-001`). Zero external cryptography libraries (no `hashlib`, `OpenSSL`, or `cryptography`), zero blockchain SDKs, zero databases/ORMs (no `sqlite3` or `sqlalchemy`), and zero web frameworks (no `flask` or `fastapi`) are used. Every core algorithm—modular arithmetic, 40-round Miller–Rabin primality testing, RSA key generation and signing, 32-bit bitwise SHA-256 hashing, domain-separated binary Merkle trees, adjustable-difficulty Proof-of-Work, Genesis-to-tip chain validation, longest-valid-chain fork resolution, UTXO hash-map indexing, and crash-safe binary persistence—is implemented and verified across 41 automated tests.

Module Path	Worklet / Layer	Core Responsibilities & First-Principles Algorithms
`blockchain_lite/interfaces.py` `blockchain_lite/serialization.py`	Shared Foundation	Cross-worklet abstract interfaces (`CanonicalSerializable`, `HashInterface`, `AsymmetricCryptoInterface`, `LedgerQueryInterface`) and deterministic big-endian `ByteBuffer` binary serializer rejecting floats, booleans, trailing bytes, and non-canonical leading zeros.
`blockchain_lite/crypto/number_theory.py` `blockchain_lite/crypto/asymmetric.py`	Worklet 1.1, 1.2 & 1.4	`mod_add`, `mod_sub`, `mod_mul`, iterative right-to-left $O(\log e)$ `pow_mod_with_trace`, Extended Euclidean GCD, `mod_inverse`, 40-round Miller–Rabin primality test (rejecting Carmichael numbers), and domain-separated deterministic `RSACryptoEngine`.

Module Path	Worklet / Layer	Core Responsibilities & First-Principles Algorithms
`blockchain_lite/models/transaction.py` `blockchain_lite/wallet/wallet.py`	Worklet 1.3, 1.4 & 1.5	`TxInput`, `TxOutput`, `Transaction` (`COINBASE` & `TRANSFER`), canonical unsigned signing preimage, intra-block/intra-tx double-spend and 1-byte tamper detection, and client-side `Wallet` isolating private keys locally.
`blockchain_lite/crypto/hashing.py` `blockchain_lite/ledger/merkle.py`	Worklet 2.2, 2.3 & 2.4	32-bit bitwise `BitLevelSHA256` (FIPS 180-4) with round-state tracer and Strict Avalanche Criterion analyzer; domain-separated binary `MerkleTree` (`0x00` leaf, `0x01` internal node) and $O(\log_2 N)$ `MerkleProof` with odd-tail phantom-index spoofing guard (`F-003`).
`blockchain_lite/models/block.py` `blockchain_lite/ledger/pow.py`	Worklet 2.1 & 2.5	88-byte fixed header `Block` model chained via `previous_hash`, crystallized Merkle/header verification, and adjustable-difficulty `ProofOfWorkMiner` searching nonces until `int(SHA256(header)) < 2^(256 - D)`.

`blockchain_lite/ledger/utxo.py`
`blockchain_lite/ledger/validator.py`
`blockchain_lite/ledger/chain.py`
	Worklet 3.1, 3.2 & 3.3	`UTXOSet` hash map (`(tx_id, output_index) -> UTXOEntry`) with atomic staged updates and coinbase reward cap; `FullChainValidator` from Genesis with optional pinned `expected_genesis_hash` (`F-001`); thread-safe `BlockchainLedger` and longest-valid-chain `resolve_forks()`.

`blockchain_lite/ledger/persistence.py` `blockchain_lite/benchmarks/pow_benchmark.py`	Worklet 3.4 & 3.5	`ChainPersistence` (`.bclt` binary format, crash-safe `.tmp` + `fsync` + `os.replace` write, 32-byte SHA-256 seal, and mandatory Genesis re-validation on reload); statistical PoW benchmark profiler.

Module Path	Worklet / Layer	Core Responsibilities & First-Principles Algorithms
`blockchain_lite/network/` `blockchain_lite/cli/demo.py`	Client-Server & Viva Demo	4-byte length-prefixed TCP socket server (`TCPSocketLedgerServer` bound to `127.0.0.1` with per-client timeout, strict schema validation, and `THR-09` private-key rejection), `WalletClient`, and 6-stage demonstration runner.

2. Canonical Binary Wire Layouts (ByteBuffer) & Security Hardening

Every domain structure implements `CanonicalSerializable` using unsigned big-endian network byte order in `ByteBuffer`. Deserialization enforces exact cursor exhaustion (`assert_exhausted()`) and rejects redundant leading zero bytes in `BigInt` encodings to eliminate transaction and block malleability.

Domain Structure	Canonical Big-Endian Binary Wire Layout
`BigInt`	`[byte_len: uint32 (4B)][unsigned_big_endian_bytes: byte_len B]` (value `0` is `0x0000000100`; leading `0x00` rejected for `byte_len > 1`)
`RSAPublicKey` / `RSAPrivateKey`	Public: `[n: BigInt][e: BigInt]` · Private: `[n: BigInt][e: BigInt][d: BigInt][p: BigInt][q: BigInt]` (validates odd n and $e \cdot d \equiv 1 \bmod \lambda(n)$)
`TxInput` (Unsigned / Signed)	Unsigned: `[prev_tx_id: 32B][output_index: uint32 (4B)][pubkey_len: uint32 (4B)][sender_pubkey: var]` · Signed appends `[sig_len: uint32][signature: var]`
`TxOutput`	`[amount: uint64 (8B)][recipient_address: 32B]` (`1 ≤ amount ≤ 10^16`; floats and booleans rejected)
`Transaction` Preimage & Full	`[tx_type: uint8 (1B)][timestamp: uint64 (8B)][in_count: uint32 (4B)]{TxInput}*[out_count: uint32 (4B)]{TxOutput}*`
`Block` Header (88 Bytes Fixed)	`[index: uint32 (4B)][previous_hash: 32B][merkle_root: 32B][timestamp: uint64 (8B)][difficulty: uint32 (4B)][nonce: uint64 (8B)]`
`MerkleProof`	`[tx_id: 32B][tx_index: uint32 (4B)][merkle_root: 32B][step_count: uint32 (4B)]{[dir: uint8 (1B)][sibling_hash: 32B]}*`
Persistence File (`.bclt`)	`[Magic: "BCLT" (4B)][Version: uint8 (1B)][Difficulty: uint32 (4B)][Reward: uint64 (8B)][BlockCount: uint32 (4B)]{[BlockLen: uint32][BlockBytes]}*[SHA256_Seal: 32B]`

3. Empirical Proof-of-Work Mining Experiment & Statistical Analysis (Worklet 3.5)

To evaluate how Proof-of-Work difficulty D (required leading zero bits in the 256-bit block header hash) governs mining cost, we benchmarked `ProofOfWorkMiner` using our pure-Python `BitLevelSHA256` engine on Python 3.11.2 (`Linux-4.19.0-gvisor-x86_64`, 2 vCPUs) across `5` independent trials per difficulty level $D \in \{0, 2, 4, 6, 8, 10\}$.

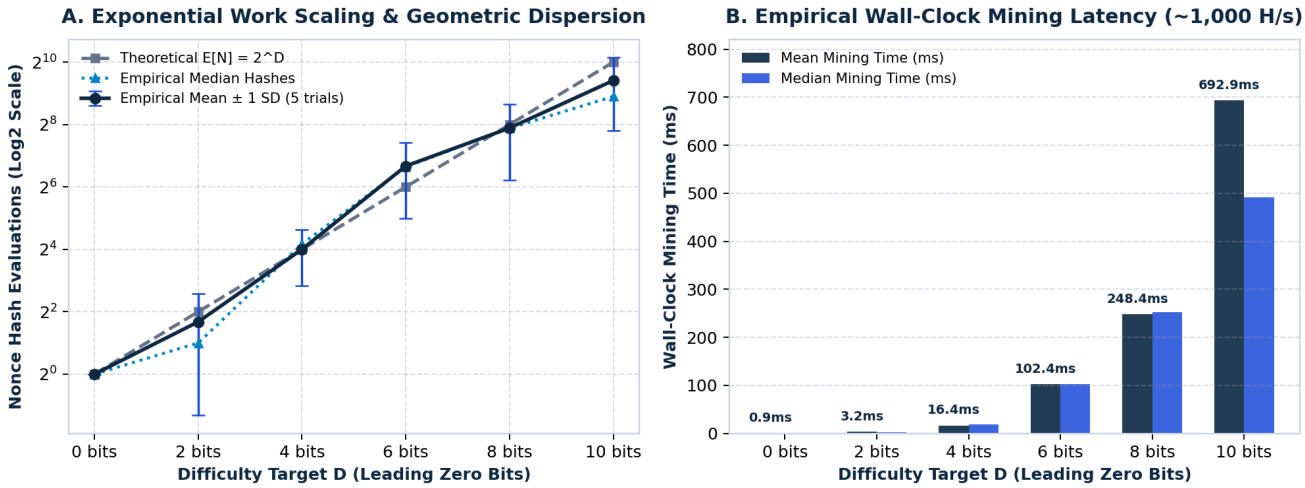


Figure 1: (A) Theoretical expected hash evaluations E[N] = 2^D vs. empirical mean (±1 SD) and median across 5 independent trials per difficulty level. (B) Corresponding mean and median wall-clock mining latency in milliseconds at ~1,000 H/s.

Difficulty (D)	Expected $E[N] = 2^D$	Theoretical $\sigma \approx 2^D$	Mean Hashes (± SD)	Median Hashes	Min..Max Hashes	Mean Time (ms)	Median Time (ms)	Throughput (H/s)
0 bits	1	0.0	1.0 (±0.0)	1.0	1..1	0.933 ms	0.903 ms	1,074 H/s
2 bits	4	3.5	3.2 (±2.8)	2.0	1..8	3.208 ms	2.011 ms	1,050 H/s
4 bits	16	15.5	16.0 (±8.9)	18.0	5..25	16.427 ms	18.420 ms	995 H/s
6 bits	64	63.5	101.8 (±70.1)	100.0	13..206	102.424 ms	102.404 ms	1,006 H/s
8 bits	256	255.5	237.0 (±162.6)	236.0	5..463	248.367 ms	252.878 ms	972 H/s
10 bits	1,024	1,023.5	680.4 (±455.7)	478.0	390..1,452	692.922 ms	491.375 ms	974 H/s

Analytical Takeaway (Why Variance Increases with Difficulty): Modeling SHA-256 as a random oracle, each candidate nonce succeeds independently with probability $p = 2^{-D}$. The number of trials N follows a Geometric distribution Geom(p) with expectation $E[N] = 1/p = 2^D$ and standard deviation $\sigma_N = \sqrt{(1 - p)}/p \approx 2^D$. Because the standard deviation grows exponentially at the same rate as the mean (coefficient of

variation $\sigma_N/E[N] \approx 1$) and the theoretical median is $\approx \ln(2) \cdot 2^D \approx 0.693 \cdot E[N]$, higher difficulty settings naturally exhibit wide trial-to-trial dispersion (e.g., `390..1,452` hashes at $D = 10$).

4. Viva Technical Reference: 5 Core Checks & All 15 Mandatory Explanations

Part A: The 5 Official COEP Viva Learning-Outcome Checks

- **Check 1 (Fast Modular Exponentiation vs. Factoring / Discrete Log):** Binary square-and-multiply (`pow_mod_with_trace`) evaluates $a^b \bmod n$ in at most $2\lfloor \log_2 b \rfloor + 1$ modular multiplications (`272` ops for a 256-bit exponent) while keeping intermediate integers below n^2. Conversely, modular reduction wraps values onto the finite ring $\mathbf{Z}_n$, destroying magnitude ordering so inverting multiplication ($n = p \cdot q$) or modular exponentiation requires sub-exponential Number Field Sieve complexity.
- **Check 2 (1-Byte Transaction Tamper Cascade):** Altering 1 byte of transaction T_j in Block B_k (verified by `trace_tamper_cascade()`) sequentially breaks: (1) the canonical signing preimage and RSA signature verification (`TamperDetectedError`); (2) `tx_id = SHA256(T_j.to_bytes())` (~50% bits flip); (3) leaf hash `H(0x00 || tx_id)` and every ancestor up to `B_k.merkle_root`; (4) `B_k.block_hash = SHA256(header_88B)` and its Proof-of-Work target; and (5) `B_{k+1}.previous_hash` across all downstream blocks $B_{k+1}..B_{\text{tip}}$.
- **Check 3 (Merkle Proof Guarantees & Asymptotic Size):** A `MerkleProof` proves to a light client holding only the 88-byte block header that `tx_id` was committed at `tx_index` inside `merkle_root`. Its size is $\Theta(\log_2 N)$ steps ($\lceil \log_2 N \rceil \times 33$ bytes + 68-byte header): 3 sibling hashes for $N = 8$ and only 20 sibling hashes (660 step bytes) for $N = 1{,}000{,}000$.
- **Check 4 (Why `previous_hash` Makes the Chain Tamper-Evident):** Because Block B_i hashes `B_{i-1}.block_hash` inside its own 88-byte PoW header, every block hash is a recursive cryptographic commitment to the entire history from Genesis. Modifying any historical block B_k alters `B_k.block_hash`, immediately breaking `B_{k+1}.previous_hash` and cascading to the tip.
- **Check 5 (Digital Signature vs. Proof-of-Work):** An RSA signature ($s = m^d \bmod n$) requires the sender's secret private key d and guarantees **authenticity, ownership authorization, and payload integrity**, but cannot prevent the owner from signing two conflicting spends of the same UTXO. Proof-of-Work requires zero secret keys and guarantees **computational cost (2^D hashes), Sybil-resistant block ordering, and objective fork convergence**, but cannot authorize spending a user's UTXOs.

Part B: Concise Answers to All 15 Phase 9 Technical Viva Questions

Technical Viva Question	Implementation-Grounded Answer
1. Why is fast modular exponentiation efficient?	It squares the base modulo m across each bit of the exponent e and multiplies only on set bits, bounding operands by m^2 and reducing time complexity from exponential $O(e)$ to logarithmic $O(\log_2 e)$.
2. What assumptions underlie RSA security?	Hardness of factoring $n = p \cdot q$ into primes p, q (needed to compute $\lambda(n) = \text{lcm}(p-1, q-1)$ and $d \equiv e^{-1} \bmod \lambda(n)$), hardness of taking e-th roots modulo composite n, and collision/preimage resistance of SHA-256.

Technical Viva Question	Implementation-Grounded Answer
3. Private key vs. password hash?	A password hash is a static digest verified by server-side comparison; a private key d stays strictly on the client and produces a unique mathematical signature $s = m^d \bmod n$ per transaction preimage, verified using only public key (n, e).
4. What exactly does the signature cover?	Domain prefix `BCLT-RSA-SHA256-V1|`, the signer's canonical `RSAPublicKey` bytes, and `Transaction.signing_preimage_bytes()` (`tx_type`, `timestamp`, all unsigned `TxInput` outpoints/pubkeys, and all `TxOutput` amounts/addresses).
5. What happens when a signed tx changes?	`signing_preimage_bytes()` changes, flipping ~50% of the SHA-256 digest bits so $s^e \bmod n$ no longer equals the recomputed message representative m'_{int}, raising `TamperDetectedError`.
6. What does the transaction ID identify?	`tx_id = SHA256(tx.to_bytes())` uniquely identifies the complete signed canonical transaction in the `UTXOSet` outpoint dictionary `(tx_id, output_index)` and in the block's Merkle tree leaves.
7. What does a Merkle proof establish?	That a specific `tx_id` was committed at leaf position `tx_index` inside a block's 32-byte `merkle_root` using $\lceil \log_2 N \rceil$ domain-separated sibling hashes without downloading all N transactions.
8. How does PoW validate a candidate block?	`verify_block_pow()` hashes the 88-byte header with `BitLevelSHA256`, confirms it equals `block.block_hash`, and checks that `int.from_bytes(block_hash, 'big') < 2^(256 - difficulty)`.
9. Why must validation start from Genesis?	Each transfer input spends a UTXO created in an earlier block; only by replaying every block from Block #0 (Genesis) can the validator verify that all referenced UTXOs exist, are unspent, and have unbroken predecessor links.
10. Why can a local ledger be rewritten by a privileged attacker?	Without an externally pinned `expected_genesis_hash` or independent network peers, an attacker with local disk write access can mine a new chain from a forged Genesis block and recompute the `.bclt` SHA-256 seal.
11. Why do balances derive from the valid chain?	There are no mutable account rows; a wallet's balance is the sum of unspent `UTXOEntry.amount` values locked to its address after replaying the active valid chain from Genesis to tip.
12. How can forks affect available balances?	Competing branches may confirm different transactions after the fork point; switching to a longer valid branch atomically replaces the `UTXOSet` with the winning branch's unspent outputs.
13. Longest valid chain vs. longest chain?	`resolve_forks()` runs `FullChainValidator` from Genesis on every candidate first, disqualifying any longer branch that contains a single bad signature, double-spend, invalid PoW, or alien Genesis block.

Technical Viva Question	Implementation-Grounded Answer
14. Client-server guarantees vs. trust?	Isolating private keys on the client guarantees the server cannot forge signatures or steal funds (`THR-09`); however, a single server is still trusted for liveness/non-censorship unless cross-checked against peers.
15. Why is this not a production blockchain?	It uses 384/512-bit deterministic educational RSA (for fast classroom keygen) rather than 2048-bit RSA-PSS/ECDSA, runs on a single loopback TCP server rather than P2P gossip, and uses pure-Python bitwise SHA-256.

5. Academic Learning-Outcome Traceability Matrix & Verification Suite

All 41 automated tests across 6 test modules (`test_phase1_foundation.py` through `test_phase6_defensive_verification.py`) pass deterministically via `python3 run_tests.py` and `python3 blockchain_lite_complete.py --test`.

Learning Outcome	Primary Source Modules	Automated Test Suites	Demo Step & Viva Focus
LO-1: Number Theory & Miller–Rabin (`REQ-001`, `REQ-002`)	`crypto/number_theory.py` `crypto/asymmetric.py`	`test_phase2_worklet1.py` `test_phase6` (Category A)	Demo Stage `[1]`: 256-bit `pow_mod` trace (272 ops) & Carmichael number rejection (561, 1105, 1729).

LO-2: RSA Signatures & Wire Format (`REQ-003`, `REQ-004`)	`serialization.py`
	`models/transaction.py`
	`wallet/wallet.py`
		`test_phase1_foundation.py`
		`test_phase2_worklet1.py`
		`test_phase6` (Category B)
			Demo Stage `[1]` & `[2]`: Client wallet creation, deterministic UTXO selection, and canonical preimage signing.

LO-3: Bit-Level SHA-256 & Avalanche (`REQ-009`)	`crypto/hashing.py`	`test_phase1_foundation.py` (`TestBitLevelSHA256`)	Demo Stage `[3]`: 1-bit input flip flips ~50% of the 256 output bits across 64 Davies–Meyer rounds.

Learning Outcome	Primary Source Modules	Automated Test Suites	Demo Step & Viva Focus
LO-4: Binary Merkle Tree & SPV Proofs (REQ-007, REQ-008)	`ledger/merkle.py`	`test_phase3_worklet2.py` `test_phase6` (Category C)	Demo Stage `[3]`: $O(\log_2 N)$ SPV inclusion proof verification & odd-tail spoofing defense (`F-003`).
LO-5: Hash Chaining & Adjustable PoW (REQ-006, REQ-010)	`models/block.py` `ledger/pow.py`	`test_phase3_worklet2.py` `test_phase6` (Category D)	Demo Stage `[2]` & `[5]`: 88-byte header mining and 1-byte historical tamper cascade trace.

LO-6: UTXO State, Validation & Forks (REQ-005, REQ-011)	`ledger/utxo.py`
	`ledger/validator.py`
	`ledger/chain.py`
		`test_phase4_worklet3.py`
		`test_phase6` (Categories D, G)
			Demo Stage `[4]` & `[5]`: Mempool/intra-block double-spend rejection & poisoned-fork rejection.

LO-7: Atomic Persistence & Reload (REQ-012, Pitfall 5)	`ledger/persistence.py`	`test_phase4_worklet3.py` `test_phase6` (Category E)	Demo Stage `[5]`: Atomic `.bclt` save/reload, outer SHA-256 seal check, and Genesis re-validation.

LO-8: TCP RPC & PoW Benchmarks (ARC-001, REQ-013)	`network/transport.py`
	`network/server.py`
	`benchmarks/pow_benchmark.py`
		`test_phase5_integration.py`
		`test_phase6` (Categories F, G)
			Demo Stage `[2]` & `[6]`: Loopback TCP client-server workflow & multi-trial PoW difficulty benchmark.